

Module 1

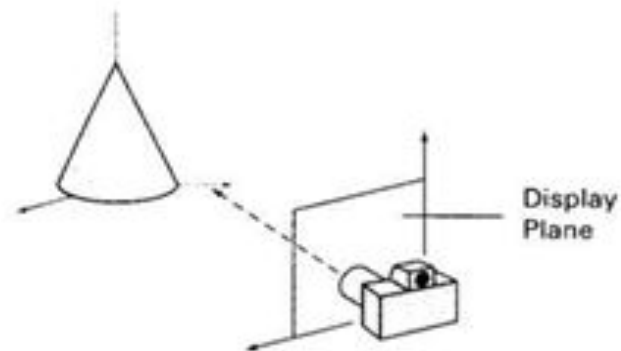
3D display methods

What is 3d display methods in computer graphics?

- 3D graphics deals with generating and displaying three-dimensional objects in a two-dimensional space.
- To generate realistic pictures, we have to first set up a coordinate reference for the camera. This coordinate reference defines the position and orientation for the plane of the camera.

- This plane used to display a view of the object
- Object description has to transfer to the camera reference co-ordinates and projected onto the selected display plane.

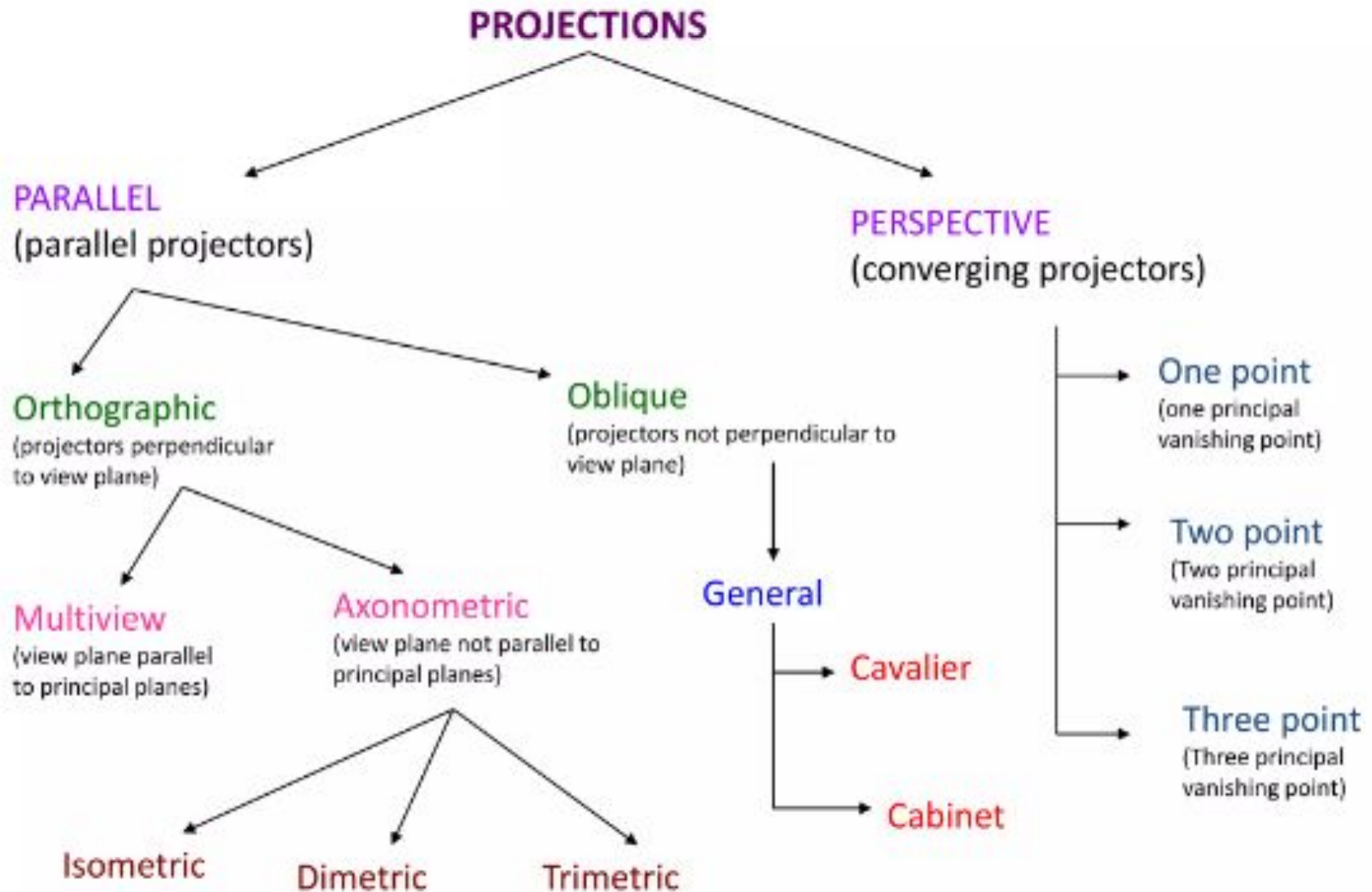
3D Display



3d display methods

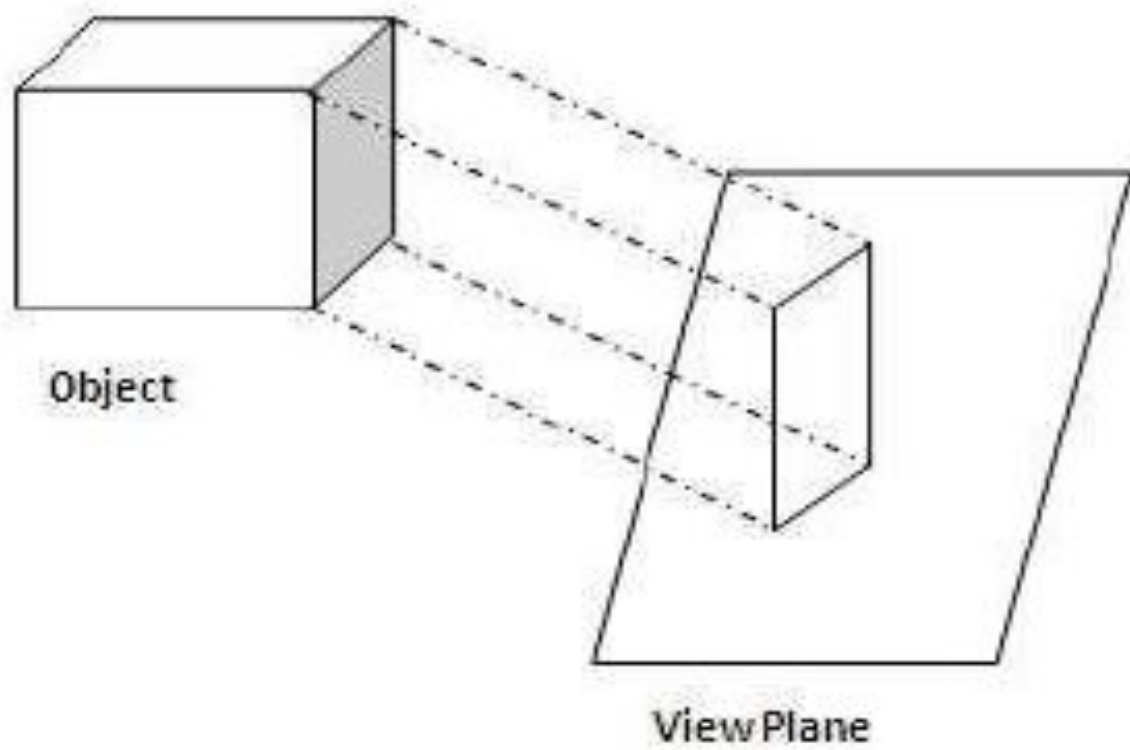
- Parallel projection
- Perspective projection
- Depth cueing
- Surface rendering
- Visible line and surface identification
- Exploded and Cutaway Views
- Three-Dimensional and Stereoscopic Views

PROJECTION



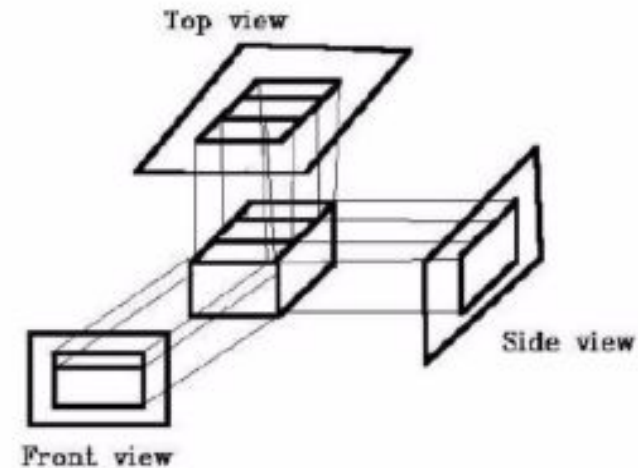
1. Parallel projection

- In parallel projection, the z-coordinate is discarded and parallel lines from each vertex on the object are extended until they intersect the view plane.
- Project points on the object surface along parallel lines onto the display plane.
- Parallel lines are still parallel after projection.
- Used in engineering and architectural drawings.
- Views maintain relative proportions of the object.



Orthographic projection

- Orthographic projection is a type of parallel projection where the direction of projection is perpendicular to the projection plane. This results in a true representation of the object's shape, but it lacks depth cues.



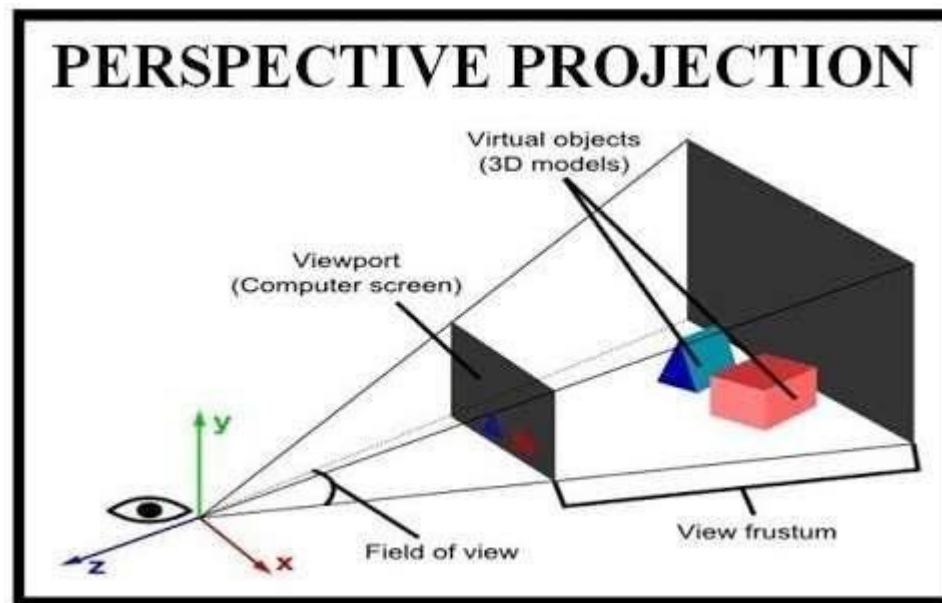
Oblique projection

- Oblique projection is a type of parallel projection where the direction of projection is not perpendicular to the projection plane. This results in a distorted representation of the object's shape, but it can provide some depth cues.

- **Cavalier projection** is a type of oblique projection where the projection lines make a 45-degree angle with the projection plane.
- **Cabinet projection** is a type of oblique projection where the projection lines make a 63.4-degree angle with the projection plane.

2. Perspective projection

- In perspective projection, the lines of projection are not parallel. Instead, they all converge at a single point called the 'center of projection' or 'projection reference point'.



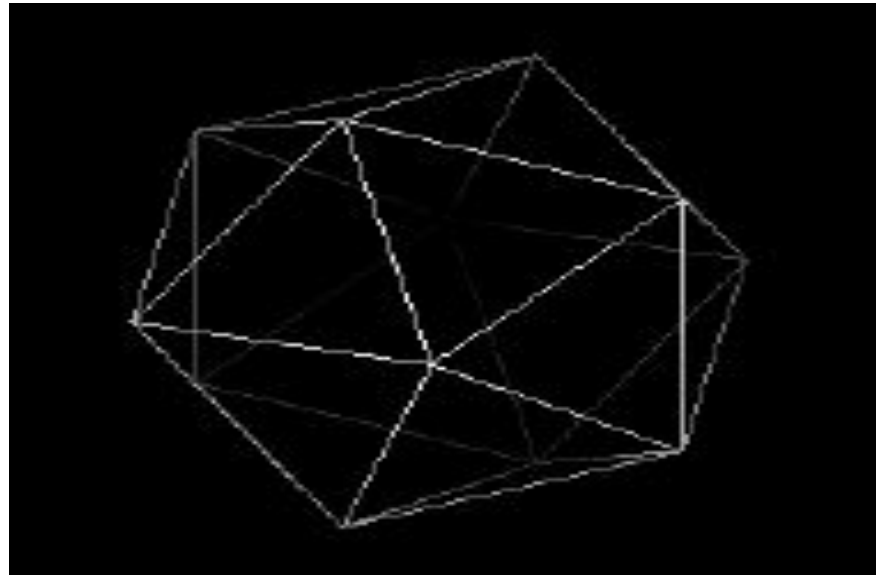
TYPES

- One point perspective projection is simple to draw. Uses one vanishing point.
- Two point perspective projection gives better impression of depth. Uses two vanishing point.
- Three point perspective projection is most difficult to draw. Uses three vanishing point.

3. Depth Cueing

- To create realistic image, the depth information is important so that we can easily identify, for a particular viewing direction, which is the front and which is the back of displayed objects.
- The depth of an object can be represented by the intensity of the image.
- The parts of the objects closest to the viewing position are displayed with the highest intensities and objects farther away are displayed with decreasing intensities. This effect is known as 'depth cueing'.

- Depth cuing is applied by choosing maximum and minimum intensity value and range of distance over which the intensities are vary.
- using it we can identify for the particular viewing direction which are the front surface and which are the back surface of the display object.



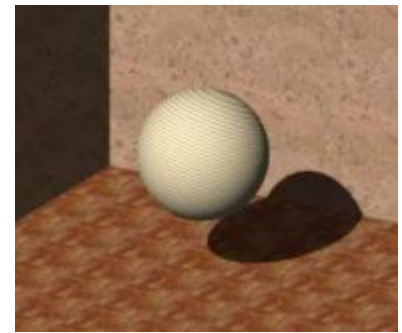
4. Visible line and surface identification

- The simplest method is to highlight the visible lines or to display them in a different colour.
- Commonly used for engineering drawing is to display the nonvisible lines as dashed lines
- Another approach is to simply remove the nonvisible lines.



- Removing hidden lines also removes information about the shape of the back surfaces of the object.
- These visible line methods also identify the visible surfaces of object.
- Visible surface identification can be done with visible surface algorithm.
- They establish visibility pixel by pixel across the viewing plane or determine surface for object as a whole.
- When objects are to be displayed with colour or shaded surface, we apply surface rendering procedures to the visible surfaces so that the hidden surfaces are obscured.

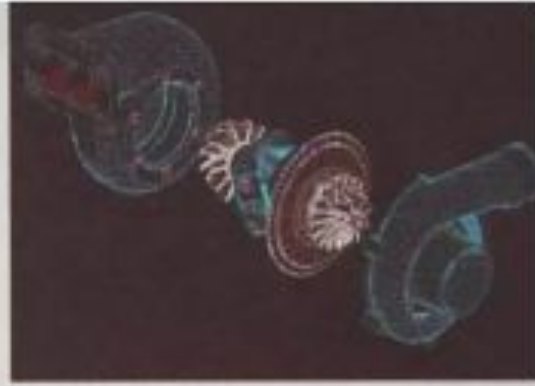
5. Surface rendering



- A method is used to generate a degree of realism in a display scene.
- Surface rendering sets the surface intensity of objects based on lighting conditions and assigned surface characteristics.
- Lighting conditions include light source intensity, positions, and general background illumination.
- Surface characteristics refer to the transparency, smoothness, and roughness of the surface.

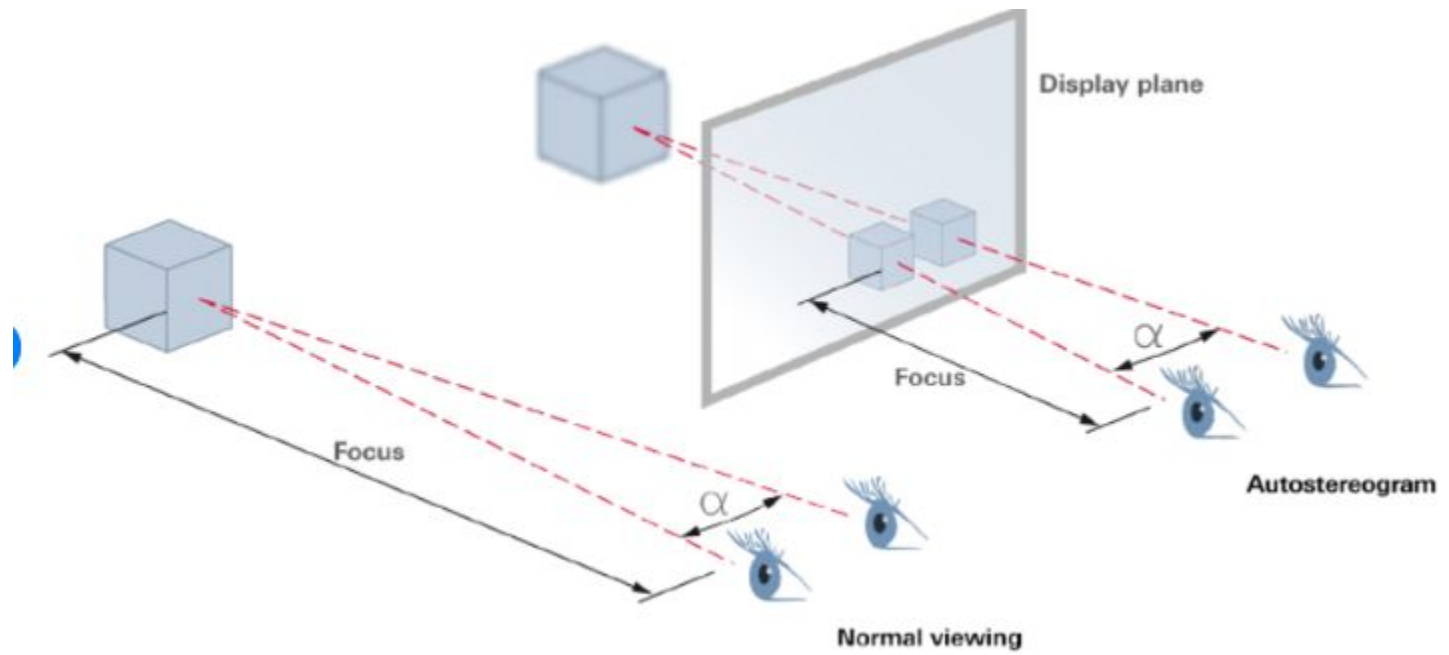
6.Exploded and Cutaway Views

- Many graphics packages allow objects to be defined as hierarchical structures, so that internal details can be stored.
- Exploded and cutaway views of such objects can then be used to show the internal structure and relationship of the object parts.



7. Three-Dimensional and Stereoscopic Views

- Three-dimensional views can be obtained by reflecting a raster image from a vibrating flexible mirror.
- Stereoscopic devices present two views of a scene: one for the left eye and the other for the right eye.
- The two views are generated by selecting viewing positions that correspond to the two eye positions of a single viewer.

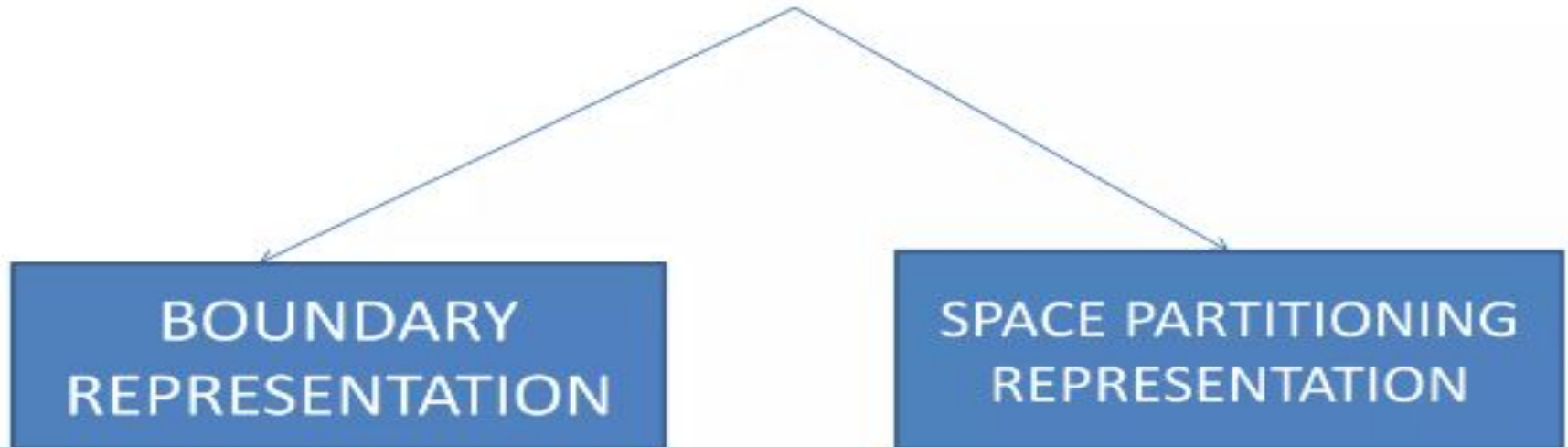


3D Object Representations

- Graphics scene contain many different kinds of objects and material surfaces. Like trees, flowers, clouds, rocks, bricks, steel, plastic, cloth etc.
- **2D Object Representation** - cannot access the proper view of an object, only gives the front view.
- **3D Object Representation** - make object more clear externally & internally, creates a proper view of any natural scene.
- Objects are represented as collection of surfaces.

Solid Object Representation -:

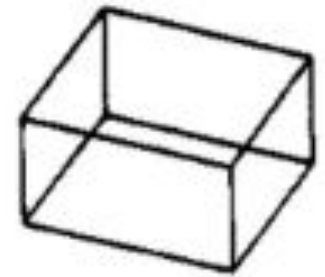
- Representation for solid objects can be divided into two categories.



- **3D object representation is divided into two categories:**

Boundary Representations (B reps)

- It describes a 3D object as a set of surfaces that separates the object interior from the environment.

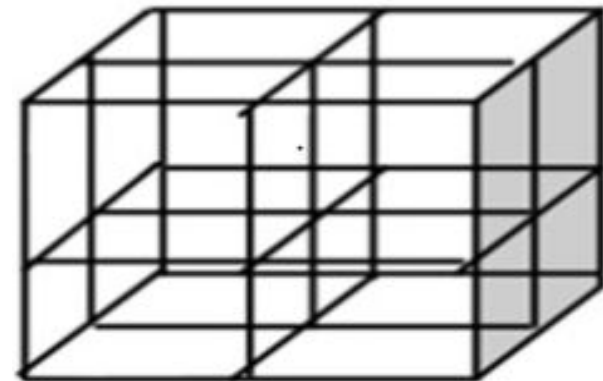


Eg: Polygon Facets, Spline Patches

Space Partitioning Representations

- It is used to describe interior properties, by partitioning the spatial region containing an object into a set of small, non-overlapping, contiguous solids (usually cubes).

Eg: Octree Representation



1. Polygon Surface representation

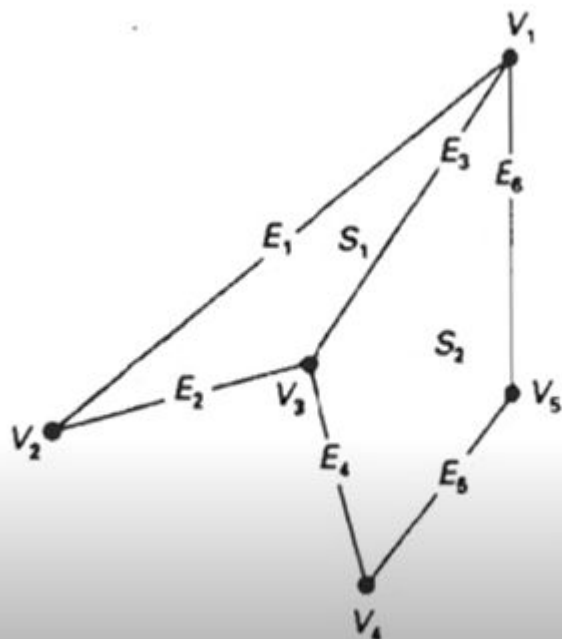
- For a three-dimensional graphics object is a set of surface polygons that enclose the object interior.
- Many graphics systems store all object descriptions as sets of surface polygons.
- This simplifies and speeds up the surface rendering and display of objects, since all surfaces are described with linear equations.
- For this reason, polygon descriptions are often referred to as "standard graphics objects."

1. Polygon Tables

- We specify a polygon surface with a set of vertex coordinates and associated attribute parameters.
- As information for each polygon is input, the data are placed into tables that are to be used in the subsequent processing, display, and manipulation of the objects in a scene.
- Polygon data tables can be organized into two groups: geometric tables and attribute tables.

- Geometric data tables contain vertex coordinates and parameters to identify the spatial orientation of the polygon surfaces.
- Attribute information for an object includes parameters specifying the degree of transparency of the object and its surface reflectivity and texture characteristics.
- A convenient organization for storing geometric data is to create three lists: a vertex table, an edge table, and a polygon table.

- Coordinate values for each vertex in the object are stored in the vertex table.
- The edge table contains pointers back into the vertex table to identify the vertices for each polygon edge.
- The polygon table contains pointers back into the edge table to identify the edges for each polygon.



VERTEX TABLE

$V_1: x_1, y_1, z_1$
 $V_2: x_2, y_2, z_2$
 $V_3: x_3, y_3, z_3$
 $V_4: x_4, y_4, z_4$
 $V_5: x_5, y_5, z_5$

EDGE TABLE

$E_1: V_1, V_2$
 $E_2: V_2, V_3$
 $E_3: V_3, V_1$
 $E_4: V_3, V_4$
 $E_5: V_4, V_5$
 $E_6: V_5, V_1$

POLYGON-SURFACE
TABLE

$S_1: E_1, E_2, E_3$
 $S_2: E_3, E_4, E_5, E_6$

- Edge slopes and bounding-box information for the polygons are needed in subsequent processing, for example, surface rendering.
- Coordinate extents are also used in some visible-surface determination algorithms.

$E_1:$	V_1, V_2, S_1
$E_2:$	V_2, V_3, S_1
$E_3:$	V_3, V_1, S_1, S_2
$E_4:$	V_3, V_4, S_2
$E_5:$	V_4, V_5, S_2
$E_6:$	V_5, V_1, S_2

Some of the tests that could be performed by a graphics package

- (1) that every vertex is listed as an endpoint for at least two edges,
- (2) that every edge is part of at least one polygon,
- (3) that every polygon is closed,
- (4) that each polygon has at least one shared edge,

2. Plane Equations

Where A,B,C,D are constants and x,y,z is any point on the plane

Equation of a polygon surface:

$$Ax + By + Cz + D = 0$$

Linear set of equations:

$$(A/D)x_k + (B/D)y_k + (C/D)z_k = -1, \quad k=1,2,3$$

By solving three plane equation for 3 non collinear points. And solve simultaneous equation for ratio A/D,B/D & C/D

- Apply Cramer's rule

$$A = \begin{vmatrix} 1 & y_1 & z_1 \\ 1 & y_2 & z_2 \\ 1 & y_3 & z_3 \end{vmatrix}$$

$$B = \begin{vmatrix} x_1 & 1 & z_1 \\ x_2 & 1 & z_2 \\ x_3 & 1 & z_3 \end{vmatrix}$$

$$C = \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}$$

$$D = - \begin{vmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{vmatrix}$$

- Expanding determinants

$$A = y_1(z_2 - z_3) + y_2(z_3 - z_1) + y_3(z_1 - z_2)$$

$$B = z_1(x_2 - x_3) + z_2(x_3 - x_1) + z_3(x_1 - x_2)$$

$$C = x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)$$

$$D = -x_1(y_2 z_3 - y_3 z_2) - x_2(y_3 z_1 - y_1 z_3) - x_3(y_1 z_2 - y_2 z_1)$$

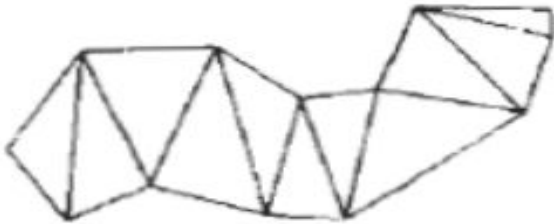
- If $Ax + By + Cz + D \neq 0$ point (X,Y,Z) is not in the plane
 surface
 - $Ax + By + Cz + D < 0$, point is inside the surface
 - $Ax + By + Cz + D > 0$, point is outside the surface

3. Polygon Meshes

- Polygon mesh is a collection of edges, vertices and faces that defines the shape of the polyhedral object in 3D computer graphics and solid modeling.
- It provide several polygon functions for modeling object
- In this method, we specify the surface facets with a mesh function.
- Main types of polygon mesh is triangle strip & quadrilateral mesh
- Edges can be shared by 2 or more polygon and vertex is shared by at least 2 edges

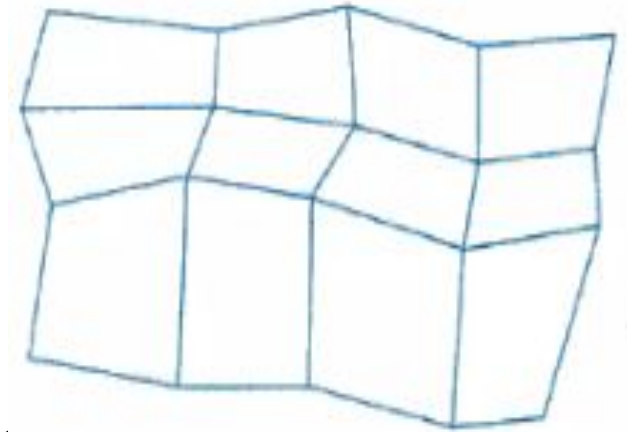
1.triangle strip

- This function produces $n - 2$ connected triangles given the coordinates for n vertices
- A triangle strip formed with 11 triangles connecting 13 vertices



2.quadrilateral mesh,

- which generates a mesh of $(n - 1)$ by $(m - 1)$ quadrilaterals, the coordinates for an n by m array of vertices.
- 20 vertices forming a mesh of 12 quadrilaterals.



Advantages

- It can be used to model almost any object.
- They are easy to represent as a collection of vertices.
- They are easy to transform.
- They are easy to draw on computer screen

Disadvantages

- Curved surfaces can only be approximately described.
- It is difficult to simulate some types of objects like hair or liquid.

CURVED LINES AND SURFACES

- Displays of 3-D curved lines and surfaces can be generated from an input set of mathematical functions defining the object or from set of user specified data points.
- When a set of co-ordinate points used to design a new object shape ,a function is obtained that fits best according to the constraint of application.eg spline curves

Curved Lines and Surfaces

- Can be represented by either **parametric or non-parametric** equations.
- Parametric more **easy to implement** in CG
- Types of curved surfaces
 - **Quadric surfaces**
 - **Superquadrics**
 - **Spline Surfaces**

Curved Lines and Surfaces

- **Regular curved surfaces** can be generated from an input set of **mathematical functions** defining objects or form a set of **user-specified data points**.
- **Mathematical Functions defining objects**
 - Quadric Surfaces (eg. Sphere, Ellipsoid)
 - Superquadrics (eg. Super ellipsoids)
- **Discrete set of coordinate points**

Functional description is obtained that **best fits** the designated points as per constraints

 - Spline curves
- Used to **design new object shapes, digitize drawings, describe animation paths, graphs of data values by curve fittings using regression techniques (least-squares method)**

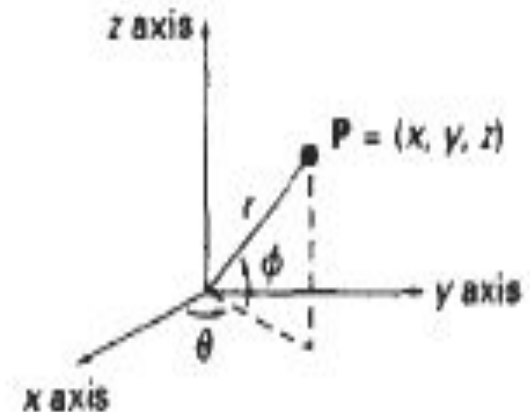
QUADRIC SURFACES

- A frequently used class of objects are the quadric surfaces, which are described with second-degree equations (quadratics)

Sphere

- A spherical surface with radius r centered on the coordinate origin is defined as the set of points (x, y, z) that satisfy the equation

$$x^2 + y^2 + z^2 = r^2$$

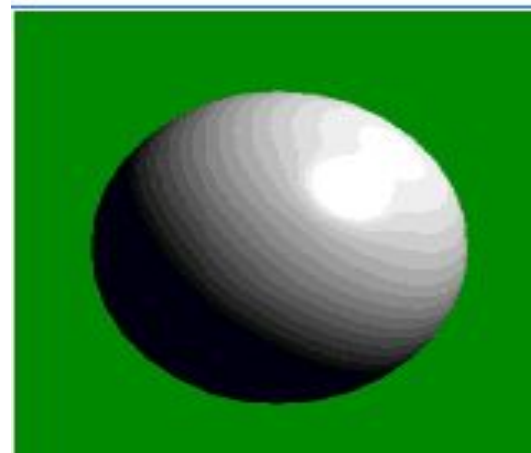


- Parametric equation using
latitude angle(ϕ) and longitude angle(θ)

$$x = r \cos \phi \cos \theta, \quad -\pi / 2 \leq \phi \leq \pi / 2$$

$$y = r \cos \phi \sin \theta, \quad -\pi \leq \theta \leq \pi$$

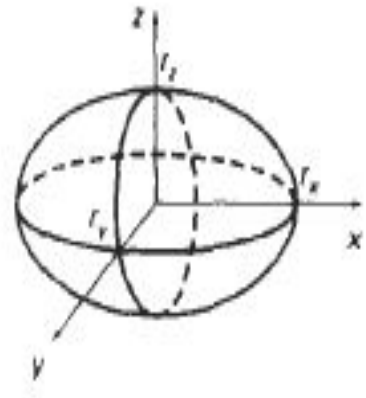
$$z = r \sin \phi$$



Ellipsoid

- An ellipsoidal surface can be described as an extension of a spherical surface, where the radii in three mutually perpendicular directions can have different values
- The Cartesian representation for points over the surface of an Ellipsoid centered on the origin is

$$\left(\frac{x}{r_x}\right)^2 + \left(\frac{y}{r_y}\right)^2 + \left(\frac{z}{r_z}\right)^2 = 1$$



- A parametric representation for the ellipsoid in terms of the latitude angle ~~ϕ~~ and the longitude angle θ .

$$\begin{aligned}x &= r_x \cos\phi \cos\theta, \\y &= r_y \cos\phi \sin\theta, \\z &= r_z \sin\phi\end{aligned}$$

Torus



- A torus is a doughnut-shaped object,
It can be generated by rotating a circle or other conic about a specified axis.

$$\left[r - \sqrt{\left(\frac{x}{r_x} \right)^2 + \left(\frac{y}{r_y} \right)^2} \right]^2 + \left(\frac{z}{r_z} \right)^2 = 1$$

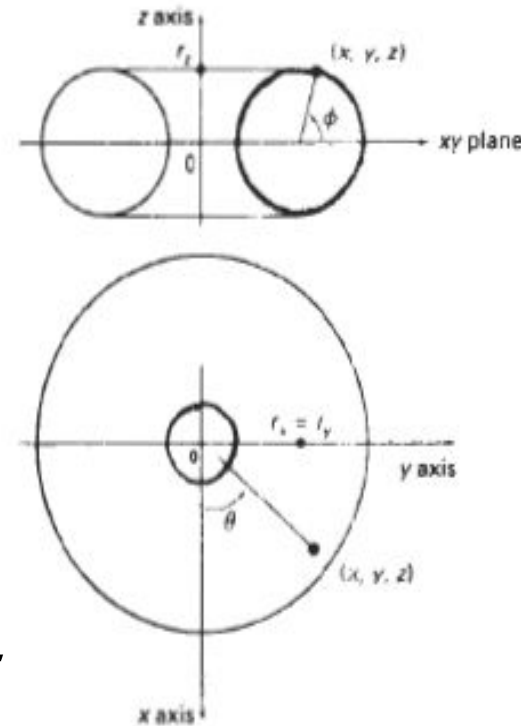
$$x = r_x (r + \cos \phi) \cos \theta$$

$$y = r_y (r + \cos \phi) \sin \theta$$

$$z = r_z \sin \phi$$

$$-\pi \leq \phi \leq \pi$$

$$-\pi \leq \theta \leq \pi$$



- where **r** is any given offset value.

SUPERQUADRICS

- This class of objects is a generalization of the quadric representations.
- **Super** quadrics are formed by incorporating additional parameters into the quadric equations to provide increased flexibility for adjusting object shapes.
- The number of additional parameters used is equal to the dimension of the object: one parameter for curves and two parameters for surfaces.

Superellipse

$$\left(\frac{x}{r_x}\right)^{\frac{2}{s}} + \left(\frac{y}{r_y}\right)^{\frac{2}{s}} = 1$$

$$x = r_x \cos^n \theta,$$

$$y = r_y \sin^n \theta$$

$$-\pi \leq \theta \leq \pi$$

- where parameter s can be assigned any real value. When $s = 1$, we get an ordinary ellipse.

Superellipsoid

$$\left[\left(\frac{x}{r_x}\right)^{2/s_2} + \left(\frac{y}{r_y}\right)^{2/s_2} \right]^{s_2/s_1} + \left(\frac{z}{r_z}\right)^{2/s_1} = 1$$

- For $s_1 = s_2 = 1$, we have an ordinary ellipsoid.

$$x = r_x \cos^{s_1} \phi \cos^{s_2} \theta,$$

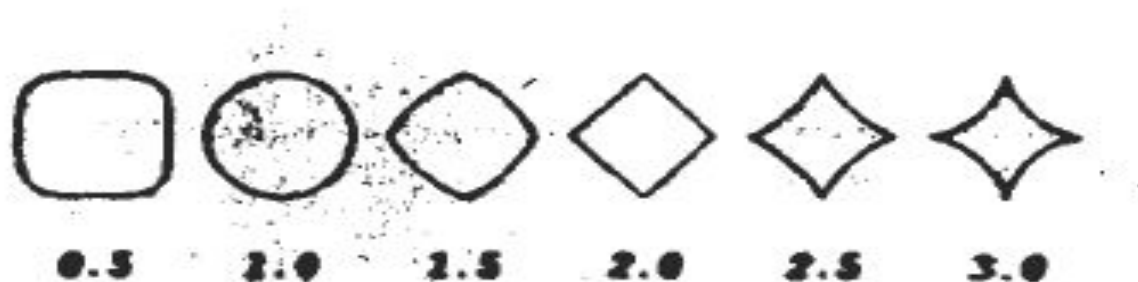
$$y = r_y \cos^{s_1} \phi \sin^{s_2} \theta,$$

$$-\pi / 2 \leq \phi \leq \pi / 2$$

$$-\pi \leq \theta \leq \pi$$

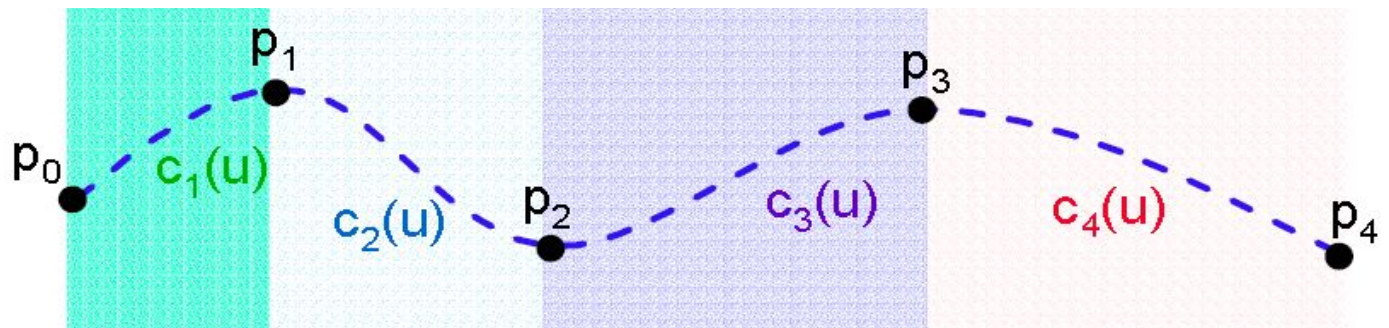
$$z = r_z \sin^{s_1} \phi$$

- Supersphere shapes that can be generated using various values for parameters s , and s_2



SPLINE REPRESENTATIONS

- A spline is a flexible strip used to produce a smooth curve through a designated set of points.
- Several small weights are distributed along the length of the strip to hold it in position on the drafting table as the curve is drawn.
- The term *spline curve* originally referred to a curve drawn in this manner. We can mathematically describe such a curve with a piecewise cubic polynomial function whose first and second derivatives are continuous across the various curve sections.

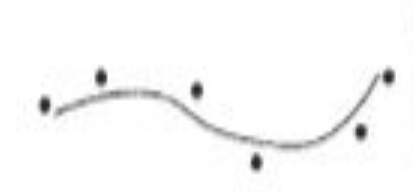


- In computer graphics, the term spline curve now refers to any composite curve formed with polynomial sections satisfying specified continuity conditions at the boundary of the pieces.
- **A** spline surface can be described with two sets of orthogonal spline curves.
- Splines are used ,in graphics applications to design curve and surface shapes, to digitize drawings for computer storage, and to do specify animation paths for the objects or the camera in a scene.
- Typical CAD applications for splines include the design of automobile bodies, aircraft and spacecraft surfaces, and ship

- A spline curve by giving a set of coordinate positions, called control points, which indicates the general shape of the curve.
- These, control points are then fitted with piecewise continuous parametric polynomial functions in one of two ways.
- There are two ways to fit a curve to these point:
 - Interpolation: the curve passes through all of the control points.



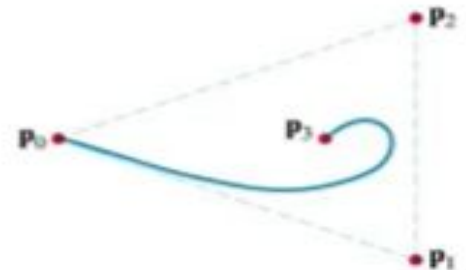
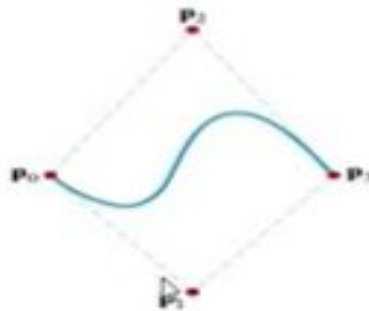
- Approximation : the curve does not necessarily pass through all of the control points.
- Interpolation curves are commonly used to digitize drawings or to specify animation paths.
- Approximation curves are primarily used as design tools to structure object surfaces.
- A spline curve is defined, modified, and manipulated with operations on the control points.
- By interactively selecting spatial positions for the control points, a designer can set up an initial curve.
- After the polynomial fit is displayed for a given set of control points, the designer can then reposition some or all of the control points to restructure the shape of the curve.



- The curve can be translated, rotated, or scaled with transformations applied to the control points.
- CAD packages can also insert extra control points to aid a designer in adjusting the curve shapes.

Convex hull

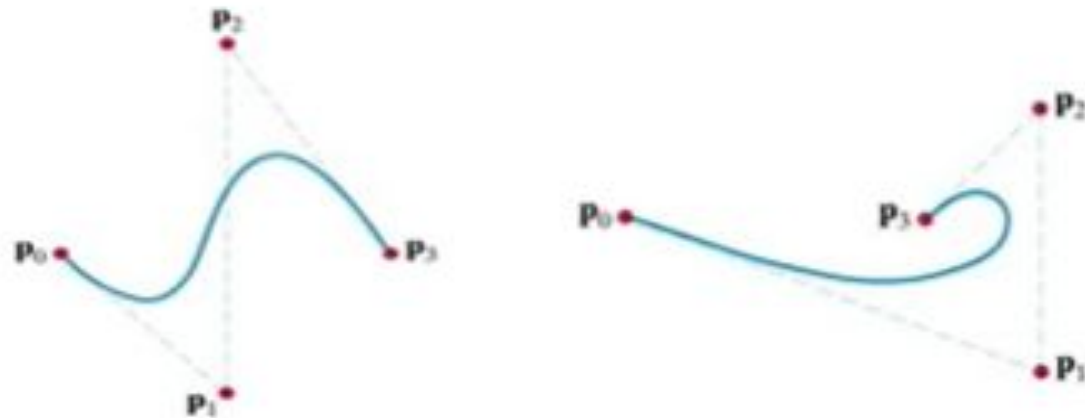
- The convex polygon boundary that encloses a set of control points is called the convex hull. ie, the boundary formed by the set of control points for a spline is known as convex hull.
- Think of an elastic band stretched around the control point



- Convexhulls provide a measure for the deviation of a curve or surface from the region bounding the control points.
- Some splines are bounded by the convex hull, thus ensuring that the polynomials smoothly follow the control points without erratic oscillations.
- The polygon region inside the convex hull is useful in some algorithms as a clipping region.

Control graph

- A polyline connecting the sequence of control points for an approximation spline are called control graph.



Parametric Continuity Conditions

- To ensure a smooth transition from one section of a piecewise parametric curve to the next, we can impose various continuity conditions at the connection points
- If each section of a spline is described with a set of parametric coordinate functions of the form
$$X=x(u), y=y(u), z=z(u) \quad u_1 \leq u \leq u_2$$
- we set parametric continuity by matching the parametric derivatives of adjoining curve sections at their common boundary.

- **Parametric continuity:** To represent a curve as a series of **piecewise parametric curves**, so that these curves will fit together reasonably. Parametric continuity is set by **matching the parametric derivation of adjoining curve sections** at their common boundary. Continuity implies a notion of **smoothness** (curves **not jagged or which break**)

- **Zero order parametric (C^0):** Simply the **curves meet**,

$$P(1) = Q(0)$$

- **First order parametric (C^1):** C^0 and curves has same tangent at intersection. The **first parametric derivatives** for two successive curve sections are **equal** at their joining point,

$$P'(1) = Q'(0)$$

- **Second order parametric (C^2):** C^0 , C^1 and both The **first and second parametric derivatives** of the two curve sections are the **same** at the intersection,

$$P''(1) = Q''(0)$$



- First-order continuity is often sufficient for digitizing drawings and some design applications.
- Second-order continuity is useful for setting up animation paths for camera motion and for many precision CAD requirements

Geometric continuity condition

- **Geometric Continuity Conditions:** Only require parametric **derivatives of the two sections to be proportional to each other** at their common boundary instead of equal to each other
- Two curve segments are said **G^k geometric continuous** at the joining point if and only if there exists two parameterizations, one for each curve segment, such that all **i -th derivatives, i less than or equal to k** , computed with these new parameterizations agree at the joining point.
- If a curve is C^0 , curves meet, it is **G^0 geometric continuous**
- Two C^0 curve segments are said **G^1 geometric continuous** at the joining point if and only if vectors **$f'(u)$ and $g'(v)$ are in the same direction** at the joining point. Note that $f'(u)$ and $g'(v)$ are evaluated at the joining point.
- Two C^1 curve segments are said **G^2 geometric continuous** at the joining point if and only if **vector $f''(u) - g''(v)$ is parallel to the tangent vector** at the joining point. Note that $f''(u)$ and $g''(v)$ are evaluated at the joining point.

•Geometric Continuity Conditions :

- Similar to parametric continuity but only the **direction of derivatives** are significant. Only require **parametric derivatives** of the two sections to be **proportional** to each other at their common boundary
- With geometric continuity, the **curve shape slightly changes**, curve is pulled toward the section with the **greater tangent vector**.
- Notion of geometric continuity provides a useful way to understand the **smoothness of a curve or surface**.
- Geometric smoothness is independent of parameterization. Parametric continuity is sufficient, but not necessary, for geometric smoothness.
- Zero order geometric (G^0):** Same as C^0 **$P(1) = Q(0)$**

- **First order geometric (G^1):** The parametric first derivatives are proportional at the intersection of two successive sections.

$$\mathbf{P}'(1) \propto \mathbf{Q}'(0)$$

$$\mathbf{P}'(1) = k\mathbf{Q}'(0)$$

The curve's tangent direction changes continuously then it is G^1 continuous.

- **Second order geometric (G^2):** Both the first and second parametric derivatives of the two curve sections are proportional at their boundary

$$\mathbf{P}''(1) \propto \mathbf{Q}''(0)$$

$$\mathbf{P}''(1) = k\mathbf{Q}''(0)$$



Show that two curves $n(t) = (t^2 + 2t - 2, t^2)$ and $r(t) = (t^2 + 2t + 1, t + 1)$ are both C^0 and G^0 continuous where they join at $n(1) = r(0)$. Do they meet C^1 and G^1 continuity.

$$\begin{aligned} n(1) &= (1^2 + 2 - 2, 1^2) \\ &= (1, 1) \\ r(0) &= (0^2 + 0 + 1, 0 + 1) \\ &= (1, 1) \end{aligned}$$

Therefore, we can say that both curves are C^0 and G^0 continuous at $n(1)$ and $r(0)$. To check for C^1 and G^1 continuity we have to take first derivative of both the curves :

$$\text{Derivative } n(t) = (2t + 2, 2t)$$

$$\text{Derivative } r(t) = (2t + 2, 1)$$

$$\therefore n(1) = (2 + 2, 2)$$

$$= (4, 2)$$

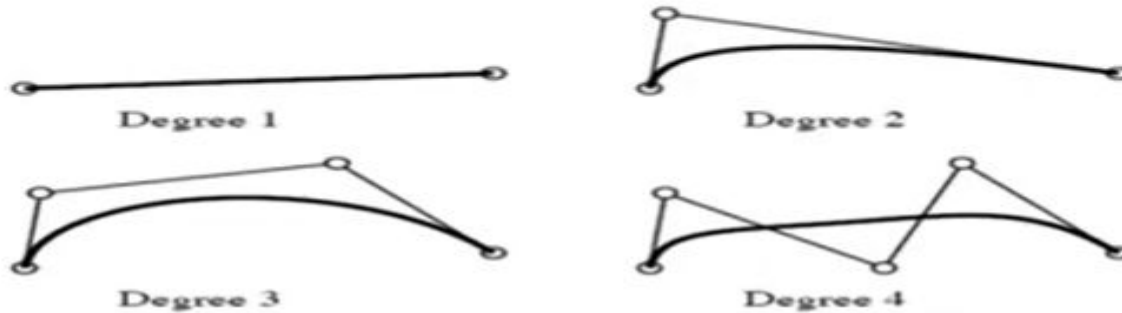
$$r(0) = (2, 1)$$

Since $n(1) \neq r(0)$, the two curves are not C^1 and G^1 continuous at $n(1)$ and $r(0)$.

BEZIER CURVES AND SURFACES

- Bezier Spline generates Bezier curve
- Easy to implement Widely used in CAD systems, graphics, drawing and painting packages.
- Can be fitted to any number of control points.
- The number of control points to be approximated and their relative position determine the **degree** of the Bezier polynomial.
- As with the interpolation splines, a Bezier curve can be specified with boundary conditions, with a characterizing matrix, or with blending functions.
- For general Bezier curves, the blending-function specification is the most convenient

Bezier curves are of different degrees which is one less than the control points



- Suppose we are given $n + 1$ control-point positions:

$$\mathbf{p}_k = (x_k, y_k, z_k),$$

with k varying from 0 to n & these coordinate points can be blended.

- These coordinate points can **be** blended to produce the following position vector **$P(u)$** , which describes the path of an approximating Bezier polynomial function between **p_0** , and **p_n** .

$$p(u) = \sum_{k=0}^n p_k BEZ_{k,n}(u) \quad 0 \leq u \leq 1$$

where

$$BEZ_{k,n}(u) = C(n, k)u^k(1 - u)^{n-k} \text{ \&}$$

$$C(n, k) = \frac{n!}{k!(n - k)!}$$

- For efficiency we can use recursive calculation of binomial coefficients as,

$$C(n, k) = \frac{n - k + 1}{k} C(n, k - 1) \quad n \geq k$$

- We can also solve Bezier blending function by recursion as,

$$BEZ_{k,n}(u) = (1 - u)BEZ_{k,n-1}(u) + uBEZ_{k-1,n-1}(u)$$

where

$$n > k \geq 1$$

$$BEZ_{k,k}(u) = u^k$$

$$BEZ_{0,k}(u) = (1 - u)^k$$

- Three parametric equations for the individual curve coordinates

$$x(u) = \sum_{k=0}^n x_k BEZ_{k,n}(u)$$

$$y(u) = \sum_{k=0}^n y_k BEZ_{k,n}(u)$$

$$z(u) = \sum_{k=0}^n z_k BEZ_{k,n}(u)$$

Properties of Bezier Curves

- It always passes through first control point i.e. $p(0) = p_0$
- It always passes through last control point i.e. $p(1) = p_n$
- Parametric first order derivatives of a Bezier curve at the endpoints can be obtain from control point coordinates as,

$$p'(0) = -np_0 + np_1$$

$$p'(1) = -np_{n-1} + np_n$$

- Parametric second order derivatives of endpoints are also obtained by control point coordinates as,

$$p''(0) = n(n-1)[(p_2 - p_1) - (p_1 - p_0)]$$

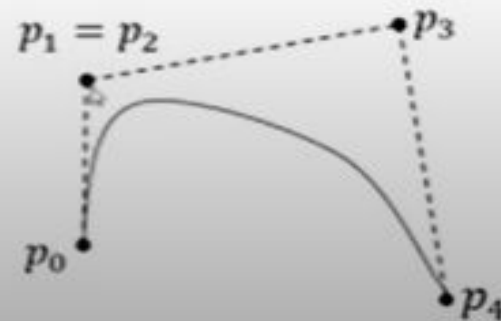
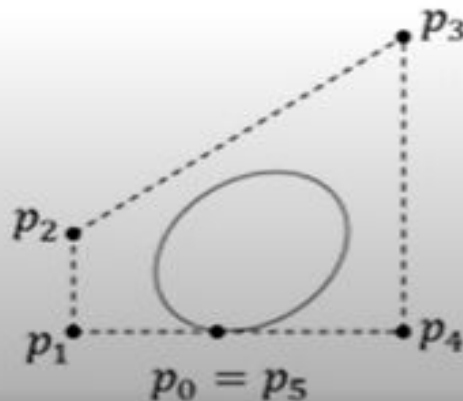
$$p''(1) = n(n-1)[(p_{n-2} - p_{n-1}) - (p_{n-1} - p_n)]$$

- Bezier curve always lies within the convex hull of the control points.
- Bezier blending function is always positive.
- Sum of all Bezier blending function is always 1.

$$\sum_{k=0}^n BEZ_{k,n}(u) = 1$$

Design Techniques using Bezier Curves

- For obtaining closed Bezier curve we specify first and last control point at same position.
- If we specify multiple control point at same position it will get more weight and curve is pull towards that position.
- Bezier curve can be fitted for any number of control points but it requires higher order polynomial calculation.

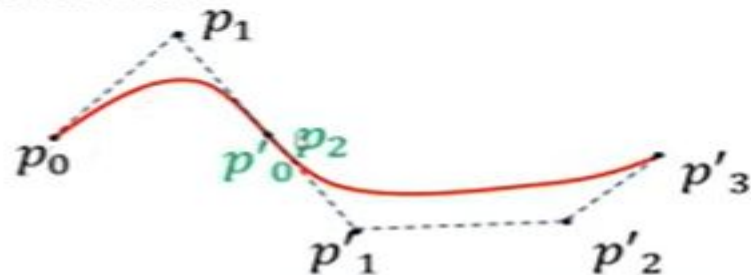


- When complicated curves are to be generated, they can be formed by piecing several Bezier sections of lower degree together.
- Piecing together smaller section also gives us better control over the shape *of* the curve in small regions.

For zero order parametric continuity (C^0)

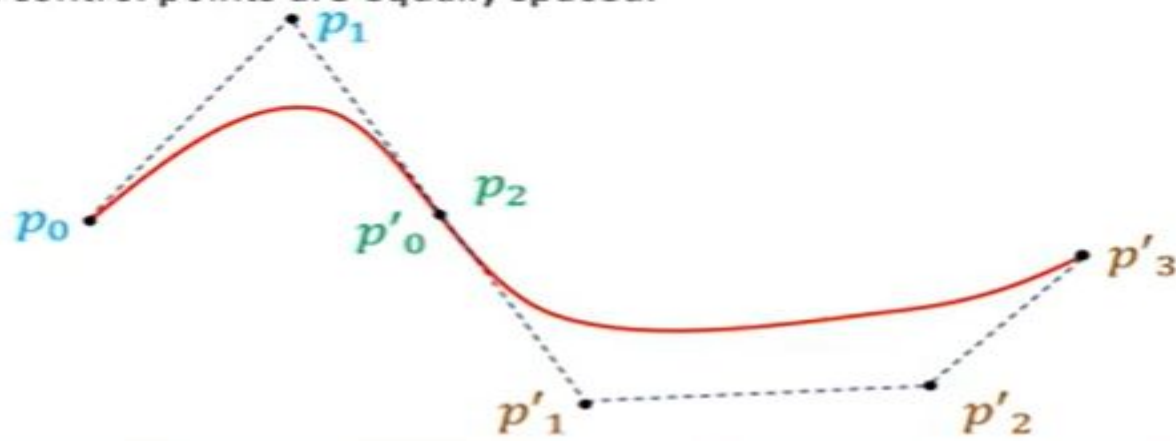
- we put end point of first curve and start point of second curve at same position.

Since Bezier curve passes through first and last control point it is easy to join two curve sections with C^0 .



For first order continuity

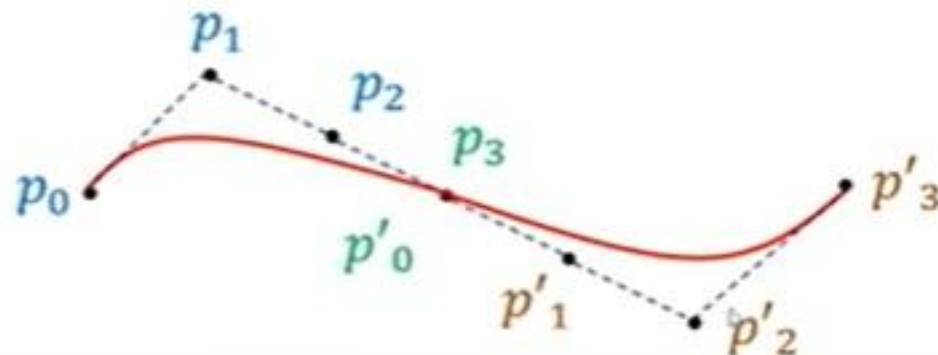
- It must be in zero order continuity
- Last two points of first curve and first two point of second curve are collinear.
- Second control point of second curve is at position $p_n + (p_n - p_{n-1})$ so that control points are equally spaced.



For second order continuity

- It must be in first order continuity
- Last three points of first curve and first three point of second curve are collinear.
- Third control point of second curve is at position $p_{n-2} + 4(p_n - p_{n-1})$ so that control points are equally spaced.

C^2 continuity can be unnecessary restrictive.



Cubic **Bezier** Curves

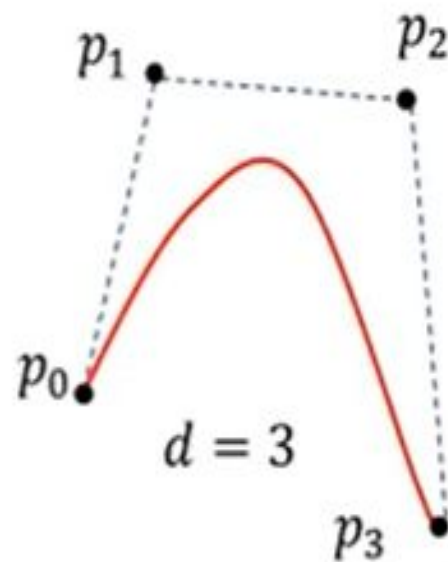
- Many graphics packages provide only cubic spline functions. This gives reasonable design flexibility while avoiding the increased calculations needed with higher-order polynomials.
- Cubic Bezier **curves** are generated with four control points.
- The four blending functions for cubic Bezier curves, obtained by substituting $n=3$

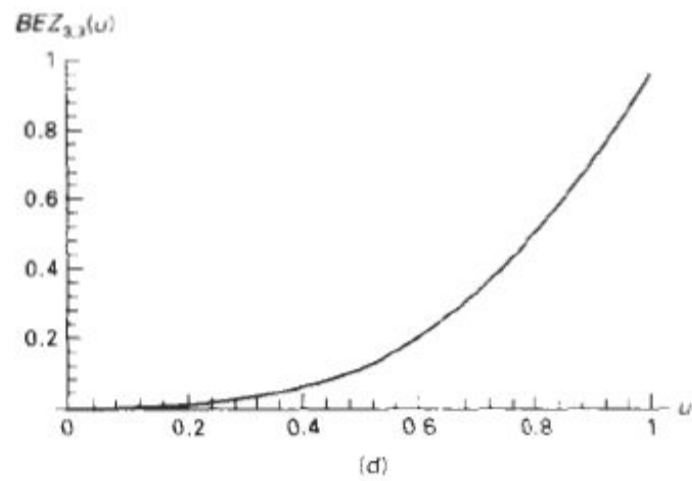
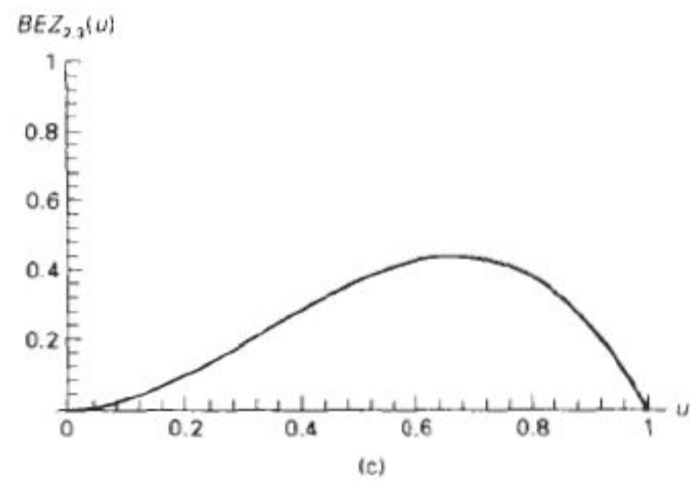
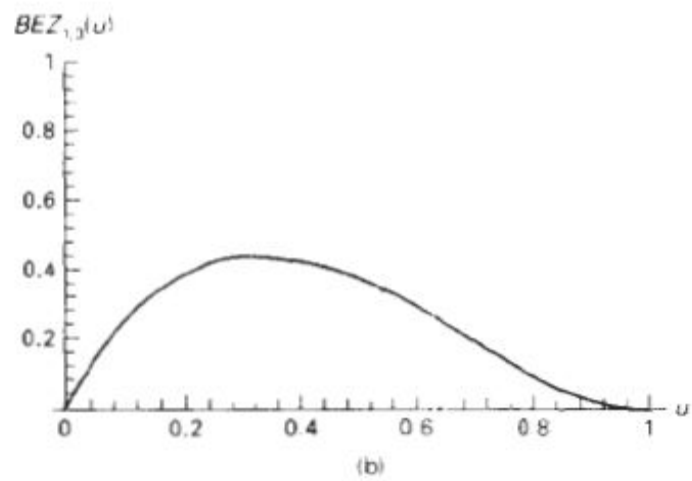
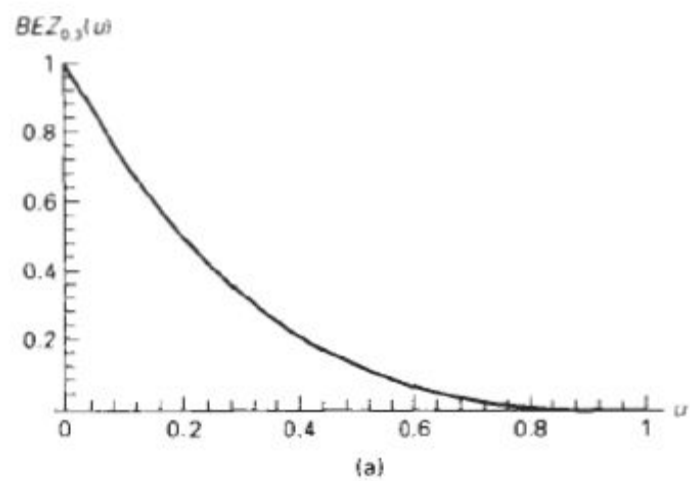
$$BEZ_{0,3}(u) = (1 - u)^3$$

$$BEZ_{1,3}(u) = 3u(1 - u)^2$$

$$BEZ_{2,3}(u) = 3u^2(1 - u)$$

$$BEZ_{3,3}(u) = u^3$$





- The form of blending functions determines how control points affect the shape of the curve for values of parameter u over the range from 0 to 1.
- At $u = 0$ $BEZ_{0,3}(u)$ is only nonzero blending function with values 1.
- At $u = 1$ $BEZ_{3,3}(u)$ is only nonzero blending function with values 1.
- So the cubic Bezier curve is always pass through p_0 and p_3 .
- Other blending function is affecting the shape of the curve in intermediate values of parameter u .
 - $BEZ_{1,3}(u)$ is maximum at $u = 1/3$
 - $BEZ_{2,3}(u)$ is maximum at $u = 2/3$

- Blending function is always nonzero over the entire range of u so it is not allowed for local control of the curve shape.
- At end point positions parametric first order derivatives are,

$$p'(0) = 3(p_1 - p_0)$$

$$p'(1) = 3(p_3 - p_2)$$
- Second order parametric derivatives are,

$$p''(0) = 6(p_0 - 2p_1 + p_2)$$

$$p''(1) = 6(p_1 - 2p_2 + p_3)$$
- This expression can be used to construct piecewise curve with C^1 and C^2 continuity.

Polynomial expression for blending function in matrix form,

$$p(u) = [u^3 \quad u^2 \quad u \quad 1] \cdot M_{BEZ} \cdot \begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

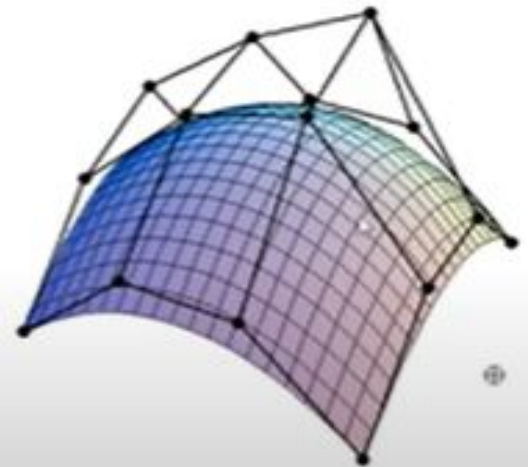
$$\text{where } M_{BEZ} = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Bezier Surfaces

- Two sets of orthogonal Bezier curves can be used to design an object surface by an input mesh of control points.
- By taking cartesian product of Bezier blending function we obtain parametric vector function as,

$$p(u, v) = \sum_{j=0}^m \sum_{k=0}^n p_{j,k} BEZ_{j,m}(v) BEZ_{k,n}(u)$$

where $p_{j,k}$ Specifying the location of the $(m + 1)$ by $(n + 1)$ control points.



- Each curve of constant u is plotted by varying v over the interval from 0 to **1**, with u fixed at one of the values in this unit interval. Curves of constant v are plotted similarly
- Bezier surfaces have the same properties as Bezier curves, and they provide a convenient method for interactive design applications.
- For each surface patch, we can select a mesh of control points in the **xy** "ground" plane, then we choose elevations above the ground plane for the z -coordinate values of the control points.
- Patches can then be pieced together using the boundary constraints.

SWEEP REPRESENTATIONS

- Solid-modeling packages often provide a number of construction techniques.
- Sweep representations are useful for constructing three-dimensional objects that possess translational, rotational, or other symmetries.
- We can represent such objects by specifying a two dimensional shape and a sweep that moves the shape through a region of space.

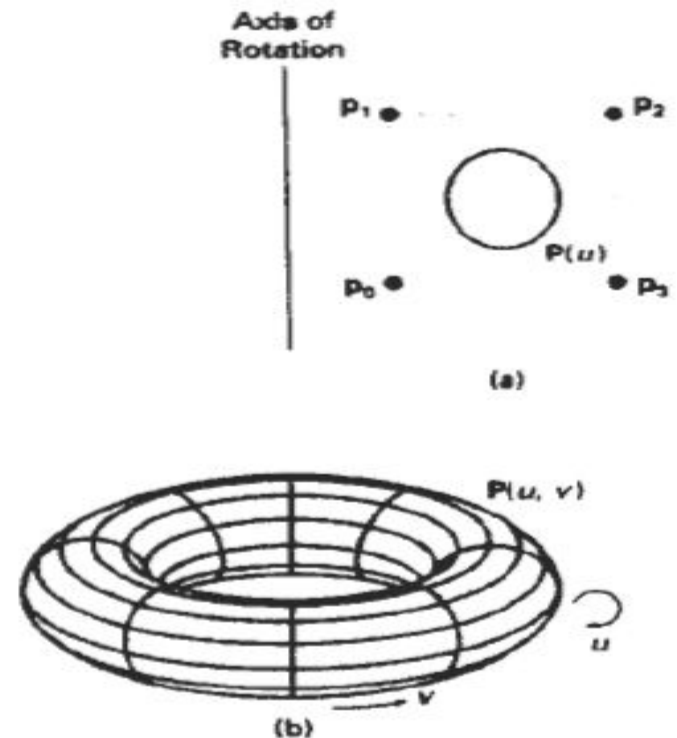
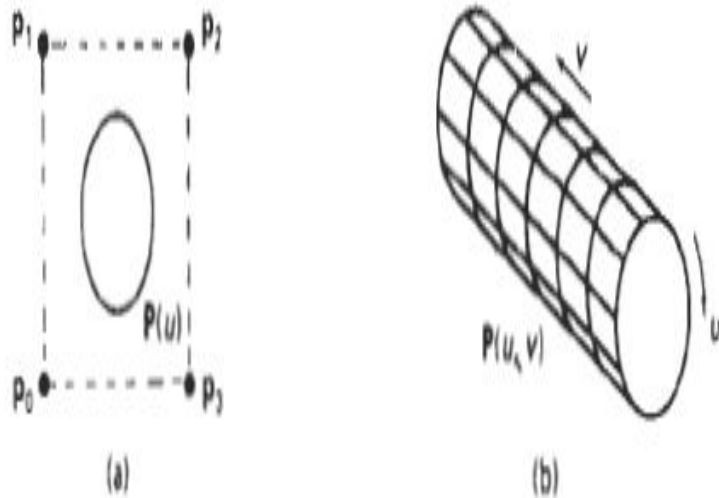
Two ways to achieve the sweep:

1. Translational sweep

The 2D shape is swept along a linear path normal to the plane of the area to construct a 3D object.

2. Rotational sweep

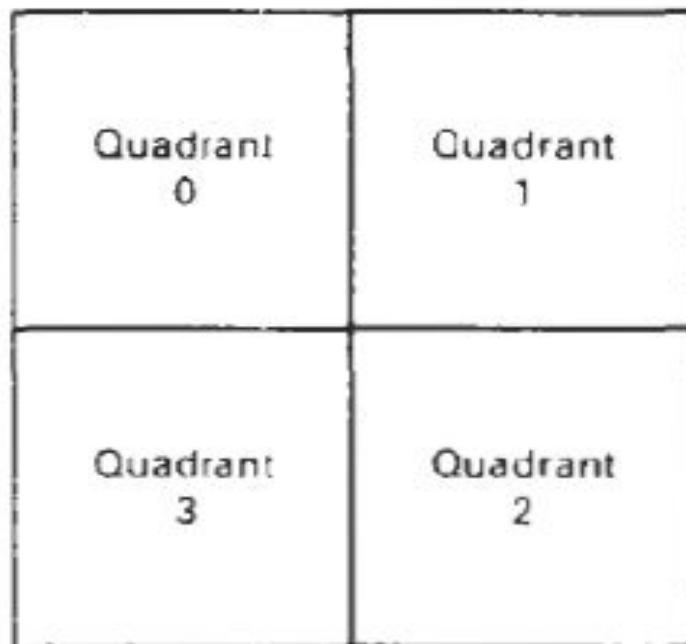
The 2D shape is rotated about an axis of rotation specified in the plane of the 2D shape to produce a 3D object.



OCTREES

- Hierarchical tree structures, called octrees, are used to represent solid objects in some graphics systems.
- Medical imaging and other applications that require displays of object cross sections commonly use **octree** representations.
- The tree structure is organized so that each node corresponds to a region of three-dimensional space.
- It also provides a convenient representation for storing information about object interiors

- The octree encoding procedure for a three-dimensional space is an extension of an encoding scheme for two-dimensional space, called quadtree encoding.
- Quadtrees are generated by successively dividing a two-dimensional region (usually a square) into quadrants.
- Each node in the quadtree has four data elements, one for each of the quadrants in the region.
- If all pixels within a quadrant have the same color (a homogeneous quadrant), the corresponding data element in the node stores that color.

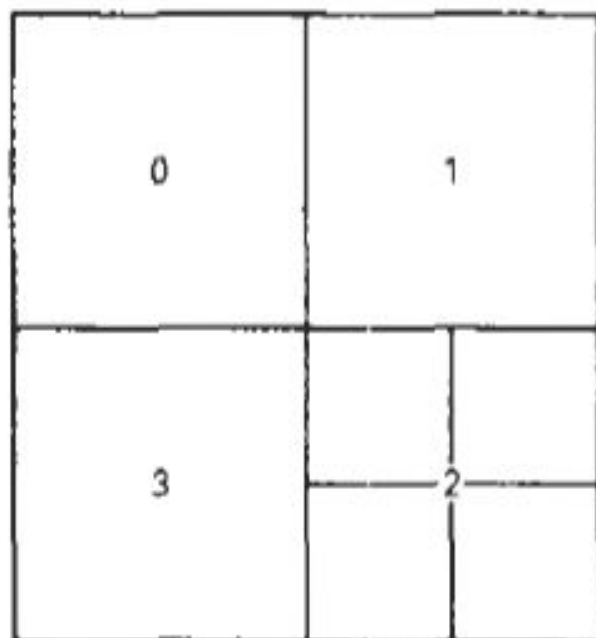


Region of a
Two-Dimensional
Space

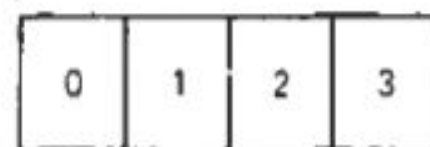
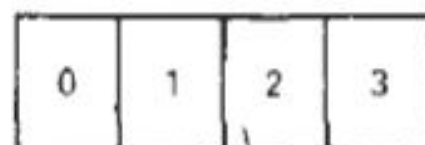
0	1	2	3
---	---	---	---

Data Elements
in the Representative
Quadtree Node

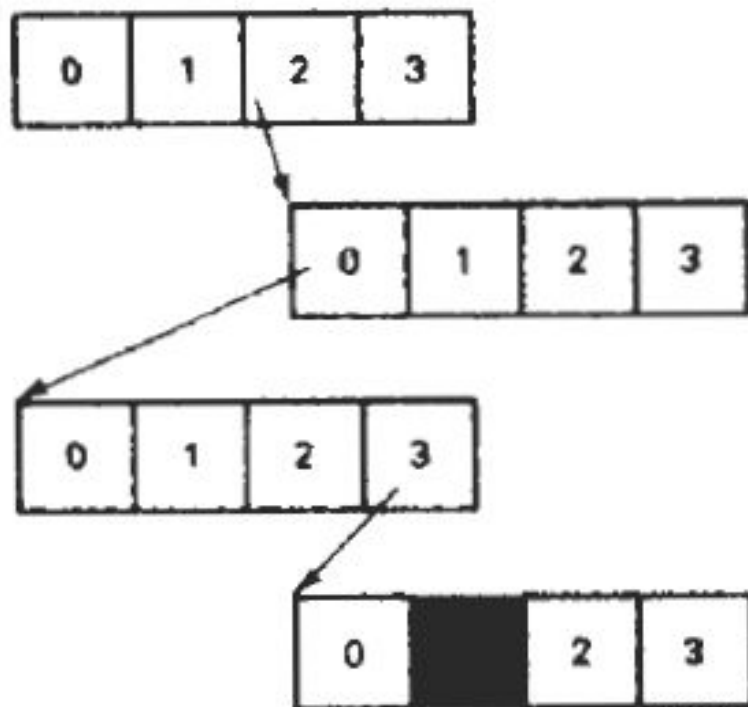
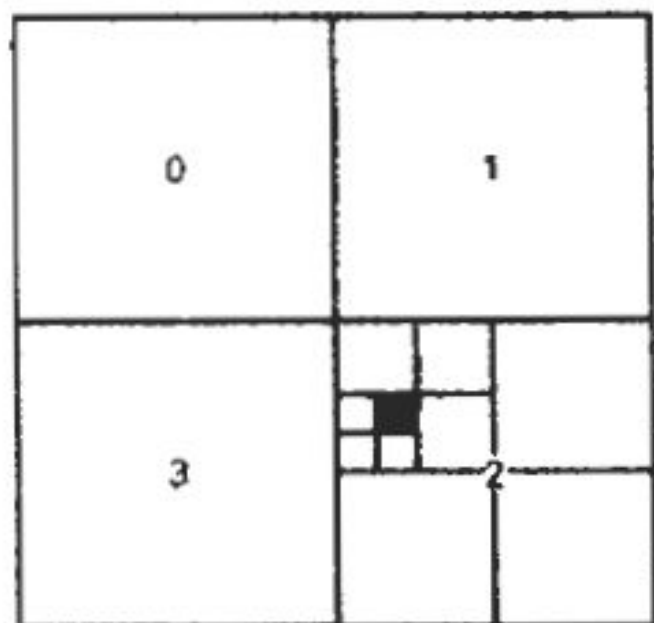
- Otherwise, the quadrant is said to be heterogeneous, and that quadrant is itself divided into quadrants.
- The corresponding data element in the node now flags the quadrant as heterogeneous and stores the pointer to the next node in the quadtree.
- For a heterogeneous region of space, the successive subdivisions into quadrants continues until all quadrants are homogeneous.



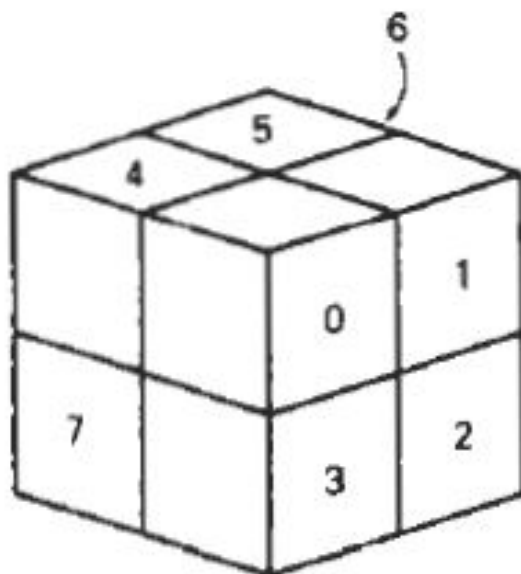
Region of a
Two-Dimensional
Space



Quadtree
Representation



- An octree encoding scheme divides regions of three-dimensional space (usually cubes) into octants and stores eight data elements in each node of the tree.
- Individual elements of a three-dimensional space are called **volume elements**, or **voxels**.
- When all voxels in an octant are of the same type, this type value is stored in the corresponding data element of the node.
- Empty regions of space are represented by voxel type "void." Any heterogeneous octant is subdivided into octants, and the corresponding data element in the node points to the next node in the octree.



Region of a
Three-Dimensional
Space

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

Data Elements
in the Representative
Octree Node